LAB 2: Node & LinkedList

[CO1]

Instructions for students:

- You may use Java / Python to complete the tasks.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN THE LINKED LIST YOU'RE CREATING.**
- **IF THE QUESTION ALLOWS YOU TO CREATE OTHER DATA STRUCTURES THEN YOU CAN.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

The Lab Tasks should be completed during the lab class
YOU HAVE TO SUBMIT ONLY THE ASSIGNMENT TASK

Total Lab 2 Assignment Tasks: 4
Total Marks: 20
Converted Marks: 4

Intro Node Tasks

This is an intro task & there isn't any driver code for this

- For java, create a separate folder for this task and follow the instructions.
- For Python, create a separate colab/ipynb/py file and follow the instructions.

➔Design a Node class.

- Declare two instance variables, one called **elem** (Object data type for java) and another one called **next** (Node datatype).
- Write a constructor that has only one parameter (Object data type for java) and assigns the parameter value to the **elem** instance variable.

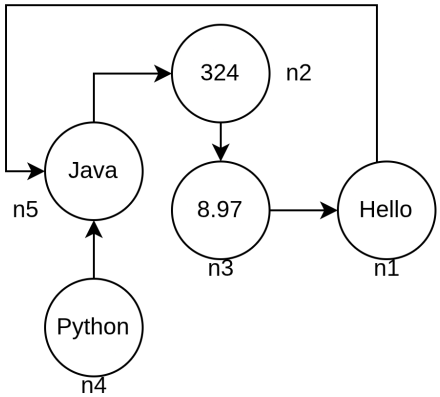
➔Design a Tester class (for java) / Design another code cell (for python)

- Create 5 different objects of Node class. Assign values as shown in the illustration.
- Variable names should be: **n1, n2, n3, n4, n5**.
- Connect the 5 nodes as shown in the illustration.

Now, execute the lines given below and try to understand the output by simulating them using pen & paper

If there are errors, try to figure out why that error occurred and what can be done about it.

Note: Java & Python output won't always be the same.



JAVA	PYTHON
`System.out.println( n1 );` `System.out.println( n3.next );`	`print( n1 )` `print( n3.next )`
`System.out.println( n3.next.elem );`	`print( n3.next.elem )`
`Node x = n4.next;` `System.out.println( n1.elem + x.elem );`	`x = n3.next` `print( x.elem + n4.elem )`
`x.next = n3;` `System.out.println(n2.next.next + n5.next);`	`x.next = n2` `print(n2.next.next + n5.next)`
`n5 = null;` `System.out.println( n4.next.elem );`	`n5 = None` `print( n4.next.elem )`
`x.next.next = null;` `n3.next.elem = 321;`	`x.next.next = None` `n2.next.elem = 7.98`
`n4.next = 532;` `System.out.println(n4.next.elem);`	`n4.next = 532` `print(n4.next.elem)`

Now let's design a basic Linked List Class.
Please use the [Java Intro LinkedList Template]() or [Python Intro LinkedList Template]() .

Intro Linked List: Design Linked list class

Node class is already given, use that from the template.

1. Complete the **append()** method inside the LinkedList class. This method takes a value as a parameter and inserts it into the back of the linked list.
    - Insert a new node at the end of the list.
    - If the list is empty, the new node becomes the head.

2. Complete the **printList()** method, which Prints all elements starting from head.
    - Traverse until the end of the list.

3. Complete the **prepend()** method inside the LinkedList class. This method takes a value as a parameter and inserts it into the front of the linked list.
    - Update the head reference properly.

4. Complete the **nodeAt()** method that returns the node at the given index (0 based).
    - If the index is invalid, return null.
    - Do NOT print inside this method.

5. Complete the **removeFirst()** method inside the LinkedList class. This should remove the first node from the linked list.
    - If the list is empty, do nothing.

6. Complete the **removeLast()** method inside the LinkedList class. This should remove the last node from the linked list.
    - If the list has only one node, head should become null.
    - If the list is empty, do nothing.

Congratulations! You have successfully designed a LinkedList class.

However, in the upcoming problem solving tasks, you will not be allowed to use this class. Instead, you will be given the head of a linked list and asked to complete specific tasks. If you need any additional helper methods, you must implement them yourself.

Now we can move on to actual problem solving using nodes/linkedlist. Please use the Java LabTask Template or Python LabTask Template

Main Lab Tasks [No Need to Submit]

1. Assemble Conga Line

Have you ever heard the term conga line? Basically, it's a carnival dance where the dancers form a long line. Everyone holds the waist of the person in front of them and their waists are held in turn by the person to their rear, excepting only those in the front and the back. It kind of looks like this.

By now, you can quite understand the suitable data structure to represent a conga line. Now you are the choreographer of the Conga Dance in a Summer Festival. You wish to arrange the conga line **ascending** age wise and tell the participants to stand in a line likewise. Now as technical you are, can you write a method that will take the conga line and return True if everyone stands according to your instruction. Otherwise returns False.

Sample Input	Sample Returned Result
10 → 15 → 34 → 41 → 56 → 72	True
10 → 15 → 44 → 41 → 56 → 72	False

2. Word Decoder

Suppose, you have been hired as an cyber security expert in an organization. A mysterious code letter has been discovered by your team, your task is to decode the letter. After lots of research you found a pattern to solve the problem. Problem details are given below:

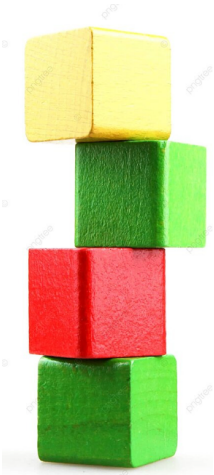
- For each encoded word a linked list will be given, where each node will carry one letter as an element.
- For the decoded word, the letters are at the multiples of $(13\ \%\ length\ of\ linkedlist)$ position **[0 indexed].** For example, if the length of the given linked list is 10, then 13 % 10 = 3 and the letters are at positions which are multiples of 3 (i.e. at position 3, 6, 9)
- However, the letters are stored in the given linked list in opposite order. Thus the decoded word has to be reversed.
- After decoding, your program should return a dummy headed singly linked list containing the decoded word.

Sample Input	Sample Output
B→M→D→T→N→O→A→P→S→C	None → C→A→T **Explanation:** Length of the list= 10 & 13%10=3 The letters are T, A, C at 3, 6, 9 positions respectively **[0 indexed].** The letters are reversed and the resulting linked list is None → C→A→T
Z→O→T→N→X	None → N **Explanation:** Length of the list= 5 & 13%5=3. The letter is N at position 3 **[0 indexed].** The letters are reversed and the resulting linked list is None → N

Assignment Tasks [Need to Submit]

1. Building Blocks

Your twin and you are under an experiment where the amount of thinking similarities you two have is being observed. As per the experiment, you are given a certain number of building blocks of different colors and are told to make a building using those blocks in two different rooms.



After the buildings are finished, the observers check whether the two buildings are the same based on the block colors. Now, you are the tech guy of that team and you are instructed to write a program that will output "Similar" or "Not Similar" given the two buildings. For fun, you decided to represent those buildings as a linked list!

NB: Red means a red block
Blue means a blue block
Yellow means a yellow block
Green means a green block.

Sample Input	Sample Output
building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green	Similar
building_1 = Red→ Green→ Yellow→ Red→ Yellow→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green	Not Similar
building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green→ Blue	Not Similar

2. Organize Books

You are a system engineer at a university library that maintains its collection of books digitally using a singly linked list. Each node in the linked list represents a book title as a string stored in the catalog.
To improve book recommendations, the library recently collected popularity scores from student surveys. Each score represents how popular a book is, a higher score means greater popularity. Your task is to reorganize the linked list so that books are arranged in descending order of popularity. If multiple books have the same popularity score, they must retain their original order.

Write a function named **organizeBooks(Node head, int[] popularity)**, which will take the **head** of the linked list and an integer array containing the **popularity** score. After organizing the list, **return the head of the modified list.**

- You can not create or take a new linked list, array or any other data structure.

Sample Given LL and Array Example1	Output
Dune → IT → Coraline → Inferno → Twilight [8, 10, 5, 10, 6]	IT → Inferno → Dune → Twilight → Coraline
<u>Explanation Example1:</u> In the given list, Dune has a score of 8, IT has 10, Coraline has 5, Inferno has 10, and Twilight has 6. The list is reorganized so that books with higher popularity appear first. Therefore, IT and Inferno, both with a score of 10, come to the front. Next is Dune with a score of 8, followed by Twilight with 6, and finally Coraline with 5.	
Sample Given LL and Array Example2	**Output2**
Hamlet → Persuasion → It → Dracula → Beloved [7, 9, 9, 6, 7]	Persuasion → It → Hamlet → Beloved → Dracula
Sample Given LL and Array Example3	**Output3**
Matilda → Franny → Foundation → Carrie → Misery [5, 8, 8, 10, 6]	Carrie → Franny → Foundation → Misery → Matilda

3. Alternate Merge

You are given two singly non-dummy headed linked lists. Your task is to write a function **alternate_merge(head1, head2)** that takes the heads of the two linked lists and returns the head of a modified linked list with all the elements of the two lists in alternate order. It is guaranteed that alternate placement is always possible. Your resulting linked list will always start with the head of linked list 1.

Input	Output
List1: 1 → 2 → 6 → 8 → 11 → None List2: 5 → 7 → 3 → 9 → 4 → None	1 → 5 → 2 → 7 → 6 → 3 → 8 → 9 → 11 → 4 → None
List1: 5 → 3 → 2 → -4 → None List2: -4 → -6 → 1 → None	5 → -4 → 3 → -6 → 2 → 1 → -4 → None
List1: 4 → 2 → -2 → -4 → None List2: 8 → 6 → 5 → -3 → None	4 → 8 → 2 → 6 → -2 → 5 → -4 → -3 → None

NOTE: The space complexity of the solution must be O(1). Which means, You're NOT ALLOWED to create any new linked list for this task.

4. ID Generator

Write a function **idGenerator()** that will take three non-dummy headed linear singly linked lists containing only digits from 0-9 and **generate another singly linked list** with 8 digit student ID from it [The student ID contains only digits from 0 to 9].

The first linked list will have the first four digits of the student id in reverse order. The remaining four digits can be obtained by adding the elements of the second linked list **from the beginning** with the elements of the third linked list **from the beginning**.

If the summation is >= 10, then you have to mod the summation by 10 and insert the result in the output linked list. For example, in sample input 2, the last element of the second list is 7 and the last element of the third linked list is 8. So after adding them we get 7+8=15>10. So the digit we will insert as an element of the Student ID list, will be 15%10=5.

Sample Input 1: 0 → 3 → 2 → 2 5 → 2 → 2 → 1 4 → 3 → 2 → 1 **Sample output 1:** 2 → 2 → 3 → 0 → 9 → 5 → 4 → 2	**Sample Input 2:** 0 → 3 → 9 → 1 3 → 6 → 5 → 7 2 → 4 → 3 → 8 **Sample output 2:** 1 → 9 → 3 → 0 → 5 → 0 → 8 → 5

You Will Have To Submit Lab 2 and Lab 3 as a single file after next week. Basically, There will be only 1 combined submission for Lab 2 & Lab 3.